

LAPORAN PRATIKUM

STRUKTUR DATA

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Dosen Pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira Putri



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman.

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan, serta kepada seluruh anggota kelompok yang telah bekerja sama dengan baik sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang,

Muhammad Yasin Habiburrahman

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1	Error! Bookmark not defined.
BAB III KESIMPULAN.....	Error! Bookmark not defined.
DAFTAR PUSTAKA.....	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dua struktur data non-linear yang paling banyak digunakan adalah Tree dan Graph. Tree atau pohon adalah struktur data hierarkis yang terdiri dari kumpulan node yang terhubung, di mana setiap node memiliki tepat satu node induk kecuali node paling atas yang disebut root. Salah satu jenis tree yang paling mendasar adalah Binary Tree, yaitu tree di mana setiap node memiliki maksimal dua anak anak kiri dan anak kanan. Binary Tree banyak digunakan dalam berbagai aplikasi seperti ekspresi matematika, pencarian data, dan kompresi data. Graph di sisi lain adalah struktur data yang terdiri dari kumpulan node (vertex) dan sisi (edge) yang menghubungkannya, tanpa aturan hierarki seperti pada tree. Graph digunakan untuk merepresentasikan berbagai permasalahan nyata seperti peta jalan, jaringan sosial, dan jaringan komputer.

1.2 Tujuan

Praktikum ini memiliki beberapa tujuan, yaitu:

1. Memahami konsep Binary Tree dan cara kerja traversal In-Order, Pre-Order, dan Post-Order.
2. Mampu mengimplementasikan Binary Tree beserta operasi pencarian dan penghitungan jumlah node menggunakan bahasa pemrograman Java.
3. Memahami konsep Graph dan perbedaannya dengan Tree dari sisi struktur dan hubungan antar node.

1.3 Manfaat Praktikum

Setelah mengikuti praktikum ini, praktikan diharapkan memperoleh manfaat sebagai berikut:

1. Praktikan mampu menerapkan Binary Tree dan Graph dalam menyelesaikan permasalahan pemrograman yang memiliki struktur hierarkis maupun relasional.
2. Praktikan memahami perbedaan pendekatan penelusuran DFS yang menjelajah sedalam mungkin sebelum berbalik, dan BFS yang menjelajah seluruh tetangga terdekat terlebih dahulu — sehingga dapat memilih algoritma yang tepat sesuai kebutuhan.

BAB II

PEMBAHASAN

2.1 Konstruktor dan Atribut Node

```
public class Node_2511532016 {
    int data_2016;
    Node_2511532016 left_2016;
    Node_2511532016 right_2016;

    public Node_2511532016(int data_2016) {
        this.data_2016 = data_2016;
        left_2016= null;
        right_2016 = null;
    }

    public void setLeft_2016(Node_2511532016 node_2016) {
        if(left_2016==null) {
            left_2016=node_2016;
        }
    }
    public void setRight_2016(Node_2511532016 node_2016) {
        if(right_2016==null) {
            right_2016=node_2016;
        }
    }
    public Node_2511532016 getLeft_2016() {
        return left_2016;
    }
    public Node_2511532016 getRight_2016() {
        return right_2016;
    }
    public int getData_2016() {
        return data_2016;
    }
    public void setData_2016(int data_2016) {
        this.data_2016=data_2016;
    }
}
```

Kode Program 2.1 – Blok Program Konstruktor, Atribut, dan Fungsi-Fungsi Pendukung

Class Node_2511532016 adalah building block dari Binary Tree, mirip seperti NodeSLL pada Single Linked List sebelumnya. Bedanya, jika node pada SLL hanya punya satu pointer next, node pada Binary Tree punya dua pointer yaitu left dan right yang masing-masing menunjuk ke anak kiri dan anak kanan. Keduanya diinisialisasi null di konstruktor karena node yang baru dibuat belum memiliki anak.

setLeft() dan setRight() menyambungkan node baru jika pointer yang bersangkutan masih null. Artinya sekali sebuah node sudah punya anak kiri atau kanan, tidak bisa diganti. Ini mencegah penimpaan node yang sudah ada secara tidak sengaja.

Terdapat pula fungsi `getData` dan `setData` untuk mengambil dan mengubah nilai yang ada pada node.

2.1.1 Algoritma-Algorithm Tree Traversal

```
void printPreorder_2016(Node_2511532016 node_2016) {
    if(node_2016 == null) return;
    System.out.print(node_2016.data_2016 + " ");
    printPreorder_2016(node_2016.left_2016);
    printPreorder_2016(node_2016.right_2016);
}

void printPostOrder_2016(Node_2511532016 node_2016) {
    if(node_2016==null) return;
    printPostOrder_2016(node_2016.left_2016);
    printPostOrder_2016(node_2016.right_2016);
    System.out.print(node_2016.data_2016 + " ");
}

void printInOrder_2016(Node_2511532016 node_2016) {
    if(node_2016==null) return;
    printInOrder_2016(node_2016.left_2016);
    System.out.print(node_2016.data_2016 + " ");
    printInOrder_2016(node_2016.right_2016);
}
```

Kode Program 2.2 – Blok Program Algoritma-Algorithm Tree Traversal

Ketiganya (Pre-Order, In-Order, Post-Order) termasuk dalam kategori Depth-First Traversal (DFT), karena ketiganya menelusuri secara rekursif sampai ke "dasar" (leaf node) sebelum berpindah ke cabang lain, mirip seperti DFS pada Graph yang sudah dibahas sebelumnya.

Bedanya hanya posisi kapan node dicetak:

Nama	Urutan	Posisi cetak
Pre-Order	Root → Kiri → Kanan	Sebelum rekursi
In-Order	Kiri → Root → Kanan	Di tengah rekursi
Post-Order	Kiri → Kanan → Root	Setelah rekursi

Tabel 2.1 – Tabel Perbandingan Algoritma Tree Traversal

2.1.2 Method untuk Menampilkan Visual Struktur Tree

```

public String print_2016() {
    return this.print_2016("", true, "");
}

public String print_2016(String prefix_2016, boolean isTail_2016, String sb_2016) {
    if(right_2016 != null) {
        right_2016.print_2016(prefix_2016 + (isTail_2016 ? "|   " : "   "), false, sb_2016);
    }
    System.out.println(prefix_2016 + (isTail_2016 ? "\\-- " : "/-- ") + data_2016);
    if(left_2016 != null) {
        left_2016.print_2016(prefix_2016 + (isTail_2016 ? "   " : "|   "), true, sb_2016);
    }
    return sb_2016;
}

```

Kode Program 2.3 – Blok Program Print

Method ini menampilkan struktur tree secara visual ke konsol menggunakan karakter \--, /--, dan | sebagai penanda cabang. Bekerja secara rekursif anak kanan dicetak terlebih dahulu di bagian atas, lalu node saat ini, lalu anak kiri di bagian bawah. Hasilnya menyerupai tree yang dirotasi 90 derajat ke kiri.

2.2 Binary Tree

Binary Tree adalah struktur data hierarkis di mana setiap node memiliki maksimal dua anak yang disebut anak kiri (left) dan anak kanan (right). Node paling atas disebut root, dan node yang tidak memiliki anak disebut leaf.

2.2.1 Atribut Binary Tree

```

private Node_2511532016 root_2016;
private Node_2511532016 currentNode_2016;

```

Kode Program 2.4 – Blok Program root dan currentNode

Class ini memiliki dua atribut, yaitu root sebagai pintu masuk ke seluruh tree, dan currentNode sebagai penunjuk node yang sedang aktif. Semua operasi pada tree selalu dimulai dari root.

2.2.2 Method-Method Pencarian

```

public boolean search (int data_2016) {
    return search(root_2016, data_2016);
}

private boolean search (Node_2511532016 node_2016, int data_2016) {
    if (node_2016.getData_2016() == data_2016) return true;
    if(node_2016.getLeft_2016() !=null) {
        if(search(node_2016.getLeft_2016(),data_2016)) return true;
    }
    if(node_2016.getRight_2016() !=null) {
        if(search(node_2016.getRight_2016(),data_2016)) return true;
    }
    return false;
}

```

Kode Program 2.5 – Blok Program

Terdapat dua versi, public dan private. Versi public hanya memanggil versi private dengan mengoper root sebagai titik awal. Versi private bekerja secara rekursif. Jika data ditemukan di node saat ini langsung return true, jika tidak rekursi ke kiri lalu ke kanan. Jika seluruh tree sudah ditelusuri dan tidak ditemukan, return false.

2.2.3 Method Menampilkan Node

```
public void printInorder_2016() {
    root_2016.printInOrder_2016(root_2016);}

public void printPreOrder_2016() {
    root_2016.printPreorder_2016(root_2016) ;}

public void printPostOrder_2016() {
    root_2016.printPostOrder_2016(root_2016) ;}
```

Kode Program 2.6 – Blok Program Untuk Melakukan Pencarian

Ketiganya hanya wrapper yang mendelegasikan pemanggilan ke method traversal milik class Node. Menarik bahwa method traversal diletakkan di class Node bukan di class BTree — keduanya sama-sama valid, hanya pilihan desain.

2.2.4 Method Menghitung Banyak Node

```
public int countNodes_2016() {
    return countNodes_2016(root_2016);
}

private int countNodes_2016(Node_2511532016 node_2016) {
    int count_2016 =1;
    if(node_2016 == null) {
        return 0;
    }
    else {
        count_2016 += countNodes_2016(node_2016.getLeft_2016());
        count_2016 += countNodes_2016(node_2016.getRight_2016());
        return count_2016;
    }
}
```

Kode Program 2.6 – Blok Program untuk Menghitung Banyak Node

Sama seperti search, terdapat dua versi public dan private. Versi private bekerja rekursif. Jika node null return 0, jika tidak return 1 + countNodes(kiri) + countNodes(kanan). Setiap node menghitung dirinya sendiri ditambah seluruh node di subtree kiri dan kanannya.

2.2.5 Mutator, Accessor, dan Validator

```
public Node_2511532016 getCurrent_2016() {
    return currentNode_2016;
}
public void setCurrent_2016(Node_2511532016 node_2016) {
    this.currentNode_2016 = node_2016;
}
public void setRoot_2016(Node_2511532016 root_2016) {
    this.root_2016 = root_2016;
}

public Node_2511532016 getRoot_2016() {
    return root_2016;
}

public boolean isEmpty_2016() {
    return root_2016 == null;
}
```

Kode Program 2.7 – Blok Program Mutator, Accessor, dan Validator untuk Kelas Binary Tree

getCurrent() dan setCurrent() digunakan untuk mengakses dan mengubah currentNode, yaitu penunjuk node yang sedang aktif di dalam tree. getRoot() dan setRoot() digunakan untuk mengakses dan mengubah root, yaitu titik masuk utama ke seluruh tree terlihat pada driver di mana setRoot() dipanggil untuk menetapkan node pertama sebagai root sebelum node-node lain ditambahkan.

isEmpty() mengecek apakah tree masih kosong dengan memeriksa apakah root bernilai null. Pada driver, method ini secara tidak langsung diuji melalui countNodes() yang dipanggil sebelum root di-set, hasilnya 0 menandakan tree memang masih kosong.

2.3 Kelas Driver Binary Tree

2.3.1 Inisialisasi Tree dan Root

```
public static void main(String[] args) {
    BTree_2511532016 tree_2016 = new BTree_2511532016();
    System.out.print("Jumlah simpul awal pohon: ");
    System.out.println(tree_2016.countNodes_2016());

    Node_2511532016 root_2016 = new Node_2511532016(1);
```

Kode Program 2.8 – Blok Program Inisialisasi Root dan Tree

Objek tree_2016 dibuat dari class BTree_2511532016. Sebelum ada node sama sekali, countNodes_2016() dipanggil dan menghasilkan 0, membuktikan tree memang kosong di awal. Node pertama root_2016 dibuat dengan nilai 1, lalu di-set sebagai root tree menggunakan setRoot_2016(). Setelah ini, countNodes_2016() menghasilkan 1 karena tree hanya berisi root.

2.3.2 Membangun Struktur Tree

```
tree_2016.setRoot_2016(root_2016);
System.out.println("Jumlah simpul jika hanya ada root");
System.out.println(tree_2016.countNodes_2016());
Node_2511532016 node2_2016 = new Node_2511532016(2);
Node_2511532016 node3_2016 = new Node_2511532016(3);
Node_2511532016 node4_2016 = new Node_2511532016(4);
Node_2511532016 node5_2016 = new Node_2511532016(5);
Node_2511532016 node6_2016 = new Node_2511532016(6);
Node_2511532016 node7_2016 = new Node_2511532016(7);
Node_2511532016 node8_2016 = new Node_2511532016(8);
Node_2511532016 node9_2016 = new Node_2511532016(9);

root_2016.setLeft_2016(node2_2016);
node2_2016.setLeft_2016(node4_2016);
node2_2016.setRight_2016(node5_2016);
node4_2016.setRight_2016(node8_2016);
root_2016.setRight_2016(node3_2016);
node3_2016.setLeft_2016(node6_2016);
node3_2016.setRight_2016(node7_2016);
node6_2016.setLeft_2016(node9_2016);
```

Kode Program 2.8 – Blok Program Contoh Struktur Tree

Delapan node lain dibuat (node2 sampai node9) dengan nilai 2-9, lalu dihubungkan satu per satu menggunakan setLeft_2016() dan setRight_2016().

2.3.3 Set Current dan Pengecekan Banyak Node

```
tree_2016.setCurrent_2016(tree_2016.getRoot_2016());
System.out.println("Menampilkan simpul terakhir");
System.out.println(tree_2016.getCurrent_2016().getData_2016());
System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
System.out.println(tree_2016.countNodes_2016());
```

Kode Program 2.9 – Blok Program Set Current dan Pengecekan Banyak Node Secara Deskriptif

currentNode_2016 di-set ke root_2016 menggunakan setCurrent_2016(). Method getCurrent_2016().getData_2016() kemudian dipanggil — namun perlu dicatat label di program tertulis "Menampilkan simpul terakhir" padahal yang ditampilkan adalah data root (nilai 1), bukan simpul terakhir. Ini kemungkinan typo komentar dari dosennya.

Setelah seluruh node ditambahkan, countNodes_2016() dipanggil dan menghasilkan 9 — sesuai jumlah total node yang sudah dibuat (root + 8 node lainnya).

2.4.2 Method Menambahkan Edge

```
public void AddEdge(String node1_2016, String node2_2016) {
    graph_2016.putIfAbsent(node1_2016, new ArrayList<>());
    graph_2016.putIfAbsent(node2_2016, new ArrayList<>());
    graph_2016.get(node1_2016).add(node2_2016);
    graph_2016.get(node2_2016).add(node1_2016);
}
```

Kode Program 2.12 – Method Menambahkan Edge pada Graph

Method ini menambahkan sisi (edge) antara dua node. `putIfAbsent()` digunakan untuk memastikan kedua node sudah punya entry di map sebelum ditambahkan tetangganya, sehingga tidak terjadi `NullPointerException`. Karena graph ini bersifat tak berarah (undirected), setiap edge ditambahkan dua arah, node1 ditambahkan ke daftar tetangga node2, dan sebaliknya.

2.4.3 Method Mencetak Graph

```
public void PrintGraph() {
    System.out.println("Graw Awal (Adjacency List) :");
    for(String node_2016 : graph_2016.keySet()) {
        System.out.print(node_2016 + " -> ");
        List<String> neighbors_2016 = graph_2016.get(node_2016);
        System.out.println(String.join(", ", neighbors_2016));
    }
    System.out.println();
}
```

Kode Program 2.12 – Blok Program untuk Mencetak Graph

Mencetak seluruh isi adjacency list. Setiap node beserta daftar tetangganya ditampilkan menggunakan `String.join(", ", neighbors)` agar tetangga-tetangga dipisahkan koma dalam satu baris.

2.4.4 Algoritma DFS

```
public void DFS_2016(String start_2016) {
    Set<String> visited_2016 = new HashSet<>();
    System.out.println("Penelusuran DFS: ");
    DFSHelper_2016(start_2016, visited_2016);
    System.out.println();
}

private void DFSHelper_2016(String current_2016, Set<String> visited_2016) {
    if(visited_2016.contains(current_2016)) return;
    visited_2016.add(current_2016);
    System.out.print(current_2016 + " ");
    for(String neighbor_2016 : graph_2016.getDefault(current_2016, new ArrayList<>())) {
        DFSHelper_2016(neighbor_2016, visited_2016);
    }
}
```

Kode Program 2.13 – Blok Program Algoritma DFS

DFS_2016() hanya inialisasi Set<String> visited lalu memanggil DFSHelper_2016() yang bekerja secara rekursif. Setiap node yang dikunjungi langsung ditambahkan ke visited dan dicetak. Kemudian untuk setiap tetangganya, dipanggil rekursi lagi jika tetangga tersebut belum dikunjungi. Pengecekan visited.contains() di awal method mencegah node yang sudah dikunjungi diproses ulang, sehingga tidak terjadi infinite loop pada graph yang punya siklus.

2.4.5 Algoritma BFS

```
public void BFS_2016(String start_2016) {
    Set<String> visited_2016 = new HashSet<>();
    Queue<String> queue_2016 = new LinkedList<>();
    queue_2016.add(start_2016);
    visited_2016.add(start_2016);

    System.out.println("Penelusuran BFS :");
    while(!queue_2016.isEmpty()) {
        String current_2016 = queue_2016.poll();
        System.out.print(current_2016 + " ");
        for(String neighbor_2016 : graph_2016.getDefault(current_2016, new ArrayList<>())) {
            if(!visited_2016.contains(neighbor_2016)) {
                queue_2016.add(neighbor_2016);
                visited_2016.add(neighbor_2016);
            }
        }
    }
    System.out.println();
}
```

Kode Program 2.14 – Blok Program Algoritma BFS

Berbeda dengan DFS yang rekursif, BFS menggunakan Queue untuk menelusuri graph per "lapisan". Node awal ditambahkan ke queue dan ditandai sudah dikunjungi. Loop while berjalan selama queue tidak kosong, setiap node diambil dari queue (poll()) dan dicetak, lalu seluruh tetangganya yang belum dikunjungi ditambahkan ke queue dan ditandai dikunjungi. Dengan cara ini, semua node pada level yang sama dikunjungi sebelum berpindah ke level berikutnya.

2.4.6 Output

```
terminated: GraphTraversal_E01155E
Graf awal adalah:
Graw Awal (Adjacency List) :
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS :
A B C D E
```

Gambar 2.14 – Output Kelas Graph Traversal

BAB III

LAPORAN TUGAS PRAKTIKUM

3.1 Deskripsi Peta yang Dibuat

Peta yang dibuat merepresentasikan alur perjalanan penumpang di sebuah Bandara Internasional, mulai dari pintu masuk terminal hingga area runway. Setiap node pada graph merepresentasikan satu titik lokasi atau fasilitas di bandara, seperti area check-in, pemeriksaan keamanan, imigrasi, area bebas pajak, gate keberangkatan, lounge, hingga runway. Sisi (edge) yang menghubungkan antar node merepresentasikan jalur fisik yang dapat dilalui penumpang untuk berpindah dari satu lokasi ke lokasi lainnya. Struktur graph ini bersifat tidak berarah karena penumpang dapat berjalan bolak balik antar lokasi yang terhubung.

3.2 Jumlah Node dan Edge

Graph yang dibuat terdiri dari 10 node (vertex) dan 17 edge, memenuhi syarat minimal yang ditentukan yaitu 10 node dan 15 edge.

Sepuluh node tersebut adalah Pintu Masuk, Check-in, Imigrasi, Security, Duty Free, Gate A, Gate B, Gate C, Lounge, dan Runway.

Tujuh belas edge yang menghubungkannya adalah Pintu Masuk-Check-in, Pintu Masuk-Imigrasi, Check-in-Security, Check-in-Imigrasi, Imigrasi-Duty Free, Security-Gate A, Security-Gate B, Security-Duty Free, Duty Free-Gate B, Duty Free-Gate C, Gate A-Lounge, Gate A-Gate B, Gate B-Lounge, Gate B-Gate C, Gate C-Runway, Lounge-Runway, dan Gate A-Runway.

3.3 Hasil BFS

Pencarian dilakukan dari titik awal Pintu Masuk ke titik tujuan Runway.

BFS menelusuri graph per level menggunakan Queue, sehingga seluruh tetangga dari satu node dikunjungi terlebih dahulu sebelum berpindah ke level berikutnya.

Jalur : Pintu Masuk -> Check-in -> Security -> Gate A -> Runway

Node Dikunjungi : Pintu Masuk -> Check-in -> Imigrasi -> Security -> Duty Free -> Gate A -> Gate B -> Gate C -> Lounge -> Runway

Jumlah Node : 10

BFS mengeksplorasi seluruh node karena Runway berada di level yang cukup dalam dari Pintu Masuk, namun jalur akhir yang ditemukan tetap merupakan jalur terpendek karena sifat BFS yang menjamin jalur dengan jumlah edge paling sedikit.

3.4 Hasil DFS

Pencarian dilakukan dari titik awal yang sama, Pintu Masuk ke titik tujuan Runway.

DFS menelusuri graph menggunakan Stack, sehingga satu cabang dijelajahi sedalam mungkin sebelum berbalik ke cabang lain.

Jalur : Pintu Masuk -> Check-in -> Security -> Gate A -> Runway

Node Dikunjungi : Pintu Masuk -> Check-in -> Security -> Gate A -> Lounge -> Gate B -> Duty Free -> Imigrasi -> Gate C -> Runway

Jumlah Node : 10

Pada kasus ini, jalur akhir yang ditemukan DFS sama persis dengan jalur BFS, karena urutan penemuan node sepanjang jalur tersebut kebetulan identik pada kedua algoritma. Namun urutan kunjungan keseluruhan node sangat berbeda, DFS langsung menyusuri cabang Gate A hingga Lounge dan Gate B terlebih dahulu sebelum kembali ke Duty Free dan Imigrasi.

3.5 Kesimpulan Perbandingan BFS dan DFS

BFS dan DFS sama-sama berhasil menemukan jalur dari Pintu Masuk ke Runway, namun dengan pola eksplorasi yang berbeda. BFS bekerja secara melebar, mengunjungi seluruh node pada level yang sama sebelum melangkah ke level berikutnya, sehingga jalur yang dihasilkan dijamin merupakan jalur terpendek dari segi jumlah langkah. DFS bekerja secara mendalam, menyusuri satu cabang sampai habis sebelum mencoba cabang lain, sehingga jalur yang ditemukan tidak selalu merupakan jalur terpendek meskipun pada kasus ini hasilnya kebetulan sama dengan BFS.

Dari segi jumlah node yang dieksplorasi, kedua algoritma sama-sama mengunjungi seluruh 10 node pada graph ini karena letak Runway yang berada cukup jauh dari Pintu Masuk. Pada graph yang lebih besar atau dengan posisi goal yang berbeda, BFS umumnya lebih cocok digunakan ketika jalur terpendek menjadi prioritas, sedangkan DFS lebih cocok ketika tujuannya hanya memastikan keterhubungan (konektivitas) antar node tanpa mempedulikan panjang jalur.

BAB IV

KESIMPULAN

Binary Tree merupakan struktur data hierarkis di mana setiap node memiliki maksimal dua anak, yaitu anak kiri dan anak kanan. Melalui praktikum ini, dapat dipahami bahwa terdapat tiga metode traversal utama pada Binary Tree, yaitu In-Order, Pre-Order, dan Post-Order, yang masing-masing memiliki urutan kunjungan node yang berbeda sesuai kebutuhan pengolahan data. Implementasi Binary Tree dalam bahasa pemrograman Java berhasil dilakukan, termasuk operasi penyisipan node, pencarian data, serta penghitungan jumlah node yang terdapat pada tree, sehingga konsep teoritis yang dipelajari dapat diterapkan secara nyata dalam bentuk program. Graph memiliki perbedaan mendasar dengan Tree, yaitu Graph tidak memiliki aturan hierarki dan terdiri dari kumpulan node (vertex) serta sisi (edge) yang dapat menghubungkan antar node secara bebas, sehingga lebih fleksibel digunakan untuk merepresentasikan hubungan antar entitas seperti jaringan sosial, peta jalan, dan jaringan komputer.

DAFTAR PUSTAKA

- [1]Oracle. *Java Documentation*. <https://docs.oracle.com/javase/>